

Chapter 1: Working with SQL in SQLite

Gregory S. DeLozier, PhD

We will begin by working directly with a database. Before a Python program or a web page sends requests on our behalf, we should be able to create some records and ask for them ourselves.

The examples come from the [introductory SQL terminal transcript](#). They use a small database of pets. We will follow the useful parts of that session, including a couple of errors and their corrections.

Structured Query Language (SQL) is the language we use to describe these database operations. SQLite is the database engine that carries them out. The `sqlite3` command gives us an interactive way to work with that engine.

Reading the Transcript

The prompt tells us where a command belongs. A line beginning with `$` is a command entered in the operating-system shell. A line beginning with `sqlite>` is input to SQLite. Continuation prompts mean the command is not finished yet. Output appears underneath without a prompt.

The prompts are part of the transcript, not part of the commands to type. Blocks labeled as transcript excerpts preserve the original session. Other code blocks contain commands prepared for this chapter.

The transcript begins with a fresh database, but the repository's `pets.db` already contains a table and records. To follow the creation steps, start in a new practice directory. These are shell commands:

```
mkdir sqlite_practice
cd sqlite_practice
sqlite3 pets.db
```

Use a new directory name for another fresh run. This keeps the existing course database available for comparison.

Opening the Database

Transcript excerpt:

```
$ pwd
/workspaces/advanced-database/chapter-01-intro-sql/code
$ sqlite3 pets.db
SQLite version 3.45.3 2024-04-15 13:34:05
Enter ".help" for usage hints.
```

`pwd` reports the working directory. The filename `pets.db` is relative to that directory, so two directories can contain different databases with that name. Pay attention to which one you opened.

Your version banner may differ from the one in the transcript. The commands below do not require that exact version.

Inspecting What Is There

Transcript excerpt:

```
sqlite> .schema
sqlite> .tables
sqlite> .headers on
sqlite> .mode column
```

At this point, there are no application tables to show. `.tables` lists tables, while `.schema` shows their definitions. `.headers on` and `.mode column` make query results easier to read.

These dot commands belong to the SQLite command-line shell. They are not SQL statements. They go on their own lines without semicolons. SQL statements in this session end with semicolons and can span several lines. The [SQLite shell documentation](#) describes both forms of input.

Creating the Pet Table

A table gives our records named columns. This is the table definition shown by `.schema` in the transcript:

```
CREATE TABLE pet
(
  id integer primary key autoincrement,
  name text not null,
  kind text not null,
  age integer,
  food text
);
```

Every pet record has the same available columns. `name` and `kind` hold text and cannot be `NULL`, the database marker for a missing value. `age` is declared as an integer. `food` holds text about what the pet eats.

The `id` column identifies a record. We do not have to supply its value in the inserts below; SQLite generates it. The transcript uses `AUTOINCREMENT`, which also explains the extra table visible in the schema output:

```
CREATE TABLE sqlite_sequence(name,seq);
```

SQLite maintains that table itself. We do not insert pet records into it. `INTEGER PRIMARY KEY` already supports automatic identifiers; `AUTOINCREMENT` adds a rule against reusing previously issued identifiers from committed rows. The distinction is documented in [SQLite's explanation of autoincrement](#).

After creating the table, the transcript checks it:

```
sqlite> .tables
pet
sqlite> .schema
```

A table definition describes what we can store. It does not create any pets yet.

Inserting a Record

The first attempt in the transcript contains a useful mistake:

```
sqlite> insert into pet (name, kind, age, food) value
('Dorothy','dog',11,'peanut butter'
(x1...> );
Parse error: near "value": syntax error
insert into pet (name, kind, age, food) value ('Dorothy','dog',11,'peanut butt
error here ----^
```

The keyword is **VALUES**, even when inserting one record. The corrected command and its result are:

```
sqlite> insert into pet (name, kind, age, food) values ('Dorothy','dog',11,'peanut butter');
sqlite> select * from pet;
id  name      kind  age  food
--  -
1   Dorothy   dog   11   peanut butter
```

The column list and value list correspond by position. Dorothy goes into name, dog goes into kind, and so on. Text values appear in single quotes; the numeric age does not need them.

Notice that the insert omits `id`, but the result includes it. SQLite supplied 1 for this new record.

Reading the Records

The transcript adds Sandy and Whiskers, then asks for the whole table:

```
sqlite> insert into pet (name, kind, age, food) values ('Sandy','cat',11,'tuna'
);
sqlite> insert into pet (name, kind, age, food) values ('Whiskers','hamster',11
,'hamster chow');
sqlite> select * from pet;
id  name      kind  age  food
--  -
1   Dorothy   dog   11   peanut butter
2   Sandy     cat   11   tuna
3   Whiskers  hamster 11   hamster chow
```

SELECT describes the values we want back. The `*` requests all columns, and **FROM** `pet` names the table.

A query result is a view of the stored data at that moment. Running this query does not remove or change those records.

Choosing Rows with WHERE

The next mistake leaves out what to select:

```
sqlite> select from pet where kind = 'cat';
Parse error: near "from": syntax error
  select from pet where kind = 'cat';
    ^--- error here
sqlite> select * from pet where kind = 'cat';
id  name  kind  age  food
--  ----  ---  ---  ---
2   Sandy  cat   11   tuna
```

The corrected statement requests all columns, but only for rows whose kind equals 'cat'. WHERE supplies the condition for including a row.

We can choose columns separately from choosing rows. Here is a command to try before deleting Sandy:

```
SELECT name, food
FROM pet
WHERE kind = 'cat';
```

This returns Sandy's name and food. It leaves out the other columns and the other pets.

Deleting Records

Transcript excerpt:

```
sqlite> delete from pet where kind = 'cat';
sqlite> select kind,food from pet;
kind      food
-----
dog       peanut butter
hamster   hamster chow
sqlite> select kind,food from pet where age = 11;
kind      food
-----
dog       peanut butter
hamster   hamster chow
```

DELETE changes the stored table. The condition selects every cat, not a particular record named Sandy. In this dataset, Sandy is the only match.

Before a deletion, a SELECT with the same condition is a useful way to inspect which records match. Leaving WHERE out of a DELETE removes every row from the table.

The remaining records still have identifiers 1 and 3. Deleting a record does not renumber the others. The last query returns both remaining pets because both have age 11 in this example.

Running a Query from the Operating-System Shell

Enter `.quit` to leave the interactive SQLite shell. The transcript later demonstrates sending a command directly from the operating-system shell:

```
$ sqlite3 pets.db ".tables"
pet
```

We can supply SQL the same way. Transcript excerpt:

```
$ sqlite3 pets.db -header -column "select * from pet where name == 'Dorothy';"
id name      kind age  food
--
1  Dorothy   dog  11   peanut butter
$ sqlite3 pets.db -header -column "select * from pet where name == 'Do';"
$ sqlite3 pets.db -header -column "select * from pet where name == 'Whiskers';"
id name      kind age  food
--
3  Whiskers  hamster 11   hamster chow
$ sqlite3 pets.db -header -column "select * from pet where name = 'Whiskers';"
id name      kind age  food
--
3  Whiskers  hamster 11   hamster chow
```

The shell's double quotes keep the SQL together as one argument. The single quotes around 'Whiskers' belong to the SQL expression. `-header -column` requests labeled, aligned output for this invocation.

The earlier session also uses `==` for equality. SQLite accepts that spelling, but we will use `=` in SQL examples.

An equality test against 'Do' does not match 'Dorothy'. An empty result is a valid answer when no stored record meets the condition.

Sending Several Lines with a Here-Document

The transcript also sends multiline input to SQLite:

```
$ sqlite3 pets.db <<'SQL'
select *
from pet
;
SQL
```

This is a shell here-document. The shell sends the following lines to the program until it reaches the closing SQL marker. The marker is not an SQL keyword in this role; it is a delimiter chosen for the shell command.

The quotes in `<<'SQL'` keep the shell from expanding variables or command substitutions in that input. The closing marker appears alone on a line.

The transcript first tries the executable name `sqlite` and gets `command not found`. The command used here is `sqlite3`.

Sorting Results

The later sorting examples show all three pets again, including Sandy:

```
$ sqlite3 pets.db -header -column "select * from pet order by kind;"
id  name      kind  age  food
--  -
2   Sandy     cat   11   tuna
1   Dorothy   dog   11   peanut butter
3   Whiskers  hamster 11   hamster chow
$ sqlite3 pets.db -header -column "select * from pet order by food;"
id  name      kind  age  food
--  -
3   Whiskers  hamster 11   hamster chow
1   Dorothy   dog   11   peanut butter
2   Sandy     cat   11   tuna
```

The transcript does not show how Sandy returned. These excerpts are useful examples, but they are not a continuous, reproducible database history. To reproduce the quoted sorting results after the deletion above, restore the original record in the practice database:

```
INSERT INTO pet (id, name, kind, age, food)
VALUES (2, 'Sandy', 'cat', 11, 'tuna');
```

This is an added step, not a transcript quotation. It supplies the original identifier explicitly so the results match the excerpt.

ORDER BY controls the order of the result. It does not rearrange the stored records or change their identifiers. Without **ORDER BY**, do not depend on the order in which a query happens to return rows.

A Small Practice Session

Use the practice database containing all three pets.

1. Display the table definition and identify the columns required by **NOT NULL**.
2. Select only the names and ages of the pets.
3. Find the pets whose age is greater than 5.
4. Show all pets ordered by name.
5. Insert another pet without supplying an identifier. Query it to see the identifier SQLite assigned.
6. Select that new pet by its identifier, then delete it using the same condition.
7. Exit SQLite and run a query from the shell to confirm that the original pets remain.

That last step connects this session to persistence. The interactive process ends, but the database file retains the records. Another invocation can retrieve them.

Questions for Thought and Study

1. How can opening `pets.db` from the wrong directory make it look as though data disappeared?
2. What distinguishes a shell command, a SQLite dot command, and an SQL statement?
3. What do the column list and value list mean in an `INSERT`?
4. How does choosing columns differ from filtering rows?
5. Why does deleting Sandy leave a gap in the identifiers?
6. How does `ORDER BY` differ from changing a record?
7. What evidence in the transcript shows that it is not one uninterrupted sequence of database changes?

Further Reading and Practice

Wikipedia

- **SQL:** Background on the language used in this chapter.
<https://en.wikipedia.org/wiki/SQL>
- **SQLite:** An overview of the database engine behind the examples.
<https://en.wikipedia.org/wiki/SQLite>

SQLite Documentation

- **Command-line shell:** Opening databases, dot commands, output formats, and running commands from the shell.
<https://www.sqlite.org/cli.html>
- **SQL language reference:** Look up the syntax of `CREATE TABLE`, `INSERT`, `SELECT`, and `DELETE`.
<https://www.sqlite.org/lang.html>
- **Automatic identifiers:** Details of `INTEGER PRIMARY KEY` and `AUTOINCREMENT`.
<https://sqlite.org/autoinc.html>

W3Schools Tutorials and Exercises

- **SQL tutorial:** Worked examples for selecting, filtering, sorting, inserting, and deleting records.
<https://www.w3schools.com/sql/>
- **SQL exercises:** Practice writing statements and checking answers.
https://www.w3schools.com/sql/sql_exercises.asp

Start with `SELECT`, `WHERE`, `ORDER BY`, `INSERT INTO`, and `DELETE`. W3Schools covers several database systems, so use the SQLite reference when syntax differs from our examples.